Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 2

«ИЗУЧЕНИЕ МЕТОДОВ ИССЛЕДОВАНИЯ ИСПОЛНЯЕМЫХ ФАЙЛОВ С ЭЛЕМЕНТАМИ САМОЗАЩИТЫ»

по дисциплине «Технологии реверс-инжиниринга программного обеспечения»

Выполнил Волошкевич М.А.
студент гр.5151001/40202 <подпись>

Преподователь / Кондачков Е.Д.
ассистент <подпись>

Санкт-Петербург
2026г.

1. Цель работы.

Изучение методов защиты, применяемых во вредоносном программном обеспечении, и способов их обхода. Изучение методов анализа бинарного кода.

2. Теоретические сведения

При анализе исполняемых файлов широко применяется ряд технологий, позволяющих автоматизировать рутинные задачи реверс-инжиниринга. К ним относится написание скриптов и плагинов к средствам дизассемблирования и отладки (IDAPython), использование средств инструментации исполняемого кода и использование технологии символьного выполнения.

IDAPython позволяет программно обходить базу дизассемблированного кода (функции, базовые блоки, инструкции, перекрёстные ссылки), что даёт возможность автоматически находить типовые паттерны самозащиты: обращения к структурам процесса (PEB/TEB), замеры времени выполнения, использование редких или привилегированных инструкций, циклы с операцией `xor`, посимвольно формируемые строки.

Символьное выполнение (в данной работе — фреймворк angr) заменяет конкретные значения входных данных символьными переменными и накапливает по ним ограничения (constraints) вдоль пути выполнения программы. Решая накопленную систему ограничений с помощью SMT-решателя, можно получить конкретные входные данные, приводящие программу в заданное состояние (например, к ветке «пароль верен»), даже если прямой анализ логики проверки вручную затруднён из-за обфускации.

3. Объект исследования

Для анализа использовались два 32-разрядных PE-файла:

Файл	**Размер**	**Точка входа**	**Назначение**
`crackme32.exe`	137 216 байт	`0x402104`	Основной crackme по индивидуальному варианту
`crackme32_1.exe`	112 640 байт	`0x4016B1`	Crackme из архива `bin2`

Таблица 1. Исследуемые исполняемые файлы

Оба файла статически слинкованы с рантаймом C/C++ (MSVC), не имеют отладочных символов и используют защиту стека (`/GS`, security cookie). Импортируемые функции обеих программ уже на этапе статического просмотра таблицы импорта указывают на присутствие в коде механизмов самозащиты: `IsDebuggerPresent`, `SetUnhandledExceptionFilter`, `UnhandledExceptionFilter`, `QueryPerformanceCounter`, `GetSystemTimeAsFileTime`; в `crackme32.exe` дополнительно присутствуют `BlockInput`, `EnumWindows`, `GetWindowTextA`, `GetClassNameA`, `MessageBoxA` — характерный набор для поиска открытых окон отладчиков по заголовку/классу окна и блокировки ввода пользователя.

4. Анализ основного файла (`crackme32.exe`)

4.1. Алгоритм проверки пароля

Программа запрашивает у пользователя строку (`Enter password:`), читает её в буфер и передаёт в функцию проверки `sub_401570`. Алгоритм функции:

1. Вычисляется длина введённой строки; если она меньше 0x25 (37) символов — проверка сразу завершается неудачей (`eax = 0`).
2. В стековый буфер копируется 38-байтовый эталонный блок, расположенный в `.rdata` по адресу `0x41EE50`.
3. Первые 38 символов введённой строки копируются в отдельный буфер (`var_2C`).
4. В цикле по адресу `0x4016C0` побайтово сравнивается преобразованный ввод (`var_2C`) с эталоном (`var_54`).
5. Если все байты совпали — функция возвращает `1` (успех), иначе `0`.

Сопоставление найденного угадываемым паролем образцов показало, что сравнение идёт не с открытым текстом, а со сдвинутой на единицу версией (посимвольный шифр Цезаря: `требуемый_символ = эталон + 1`) — например, фрагменту эталона `Sebret`/`Backdnnr`/`Passvnrd` соответствуют введённые `Secret`/`Backdoor`/`Password`. Это и есть простейшее «кодирование строк», которое требовалось найти по заданию.

Адрес	**Назначение**
`0x401570`	Начало функции проверки пароля (`sub_401570`)
`0x4016C0`	Цикл побайтового сравнения преобразованного ввода с эталоном
`0x401C35`	Точка вызова `sub_401570` из `main`
`0x401C3D`–`0x401C3F`	`test eax, eax` / `je 0x401C7A` — ветвление по результату проверки
`0x401C59`–`0x401C75`	Ветка успеха: вычисление и вывод «секретного кода»

Таблица 2. Ключевые адреса алгоритма проверки



Рис. 1. Окно IDA Pro: граф функции sub_401570 (алгоритм проверки пароля)

4.2. Защитные механизмы

Статическим поиском (по дизассемблированному листингу и таблице импорта) в `crackme32.exe` обнаружены следующие категории признаков самозащиты.

Категория	**Конкретные находки**	**Принцип действия**
Проверка отладчика	`IsDebuggerPresent`, `SetUnhandledExceptionFilter`	Сверка `UnhandledExceptionFilter`/ флага `BeingDebugged` в PEB и перехват необработанных исключений как канал анти-отладки
Обращение к PEB	`mov eax, fs:0x30` (адрес `0x40A379`)	Прямое чтение адреса PEB из TEB (`fs:[0x30]` в 32-битном процессе) в обход высокоуровневого API
Замер времени	`rdtsc` (`0x40120F`, `0x401237`), `QueryPerformanceCounter`	Измерение числа тактов/времени выполнения участка кода: аномально большое время говорит о пошаговой трассировке в отладчике
Редкие/привилегированные инструкции	`int 0x2D` (`0x40117A`), `cpuid` (`0x40274C` и др.), префикс `bnd` (MPX, несколько адресов)	`int 0x2D` — классический anti-debug триггер (особая обработка в ядре при отсутствии отладчика); `cpuid`/`bnd` затрудняют линейный дизассемблинг и используются для fingerprinting CPU

Поиск окна отладчика	`EnumWindows`, `GetWindowTextA`, `GetClassNameA`, `BlockInput`	Перечисление окон верхнего уровня и сравнение заголовка/класса с окнами известных отладчиков (OllyDbg, x64dbg, IDA); `BlockInput` блокирует ввод, мешая взаимодействию с отладчиком
Анти-VM / fingerprint окружения	Строки `"Checking reg key %s: ..."`, `"HARDWARE\Description\System"`	Чтение значений реестра, типично используемых для определения имени CPU/BIOS и выявления виртуальной машины или песочницы
Защита от модификации/отладки через исключения	`int 0x29` (несколько адресов), инструкции `bnd ret`/ `bnd jne`	`int 0x29` — быстрый отказ (fast-fail), используемый, в том числе, проверкой стекового канарейки (`/GS`); `bnd`-префиксы относятся к Intel MPX и также являются стандартной компиляторной защитой, не специфичной для данного crackme

Таблица 3. Обнаруженные индикаторы защитных механизмов

Ïðèì÷àíèå: ÷àñòü íàõîäîê (`rdtsc`, `cpuid`, `int 0x29`, `bnd`) îòíîñèòñÿ ê ñòàíäàðòíîìó êîäó, ãåíåðèðóåìîìó êîìïèëÿòîðîì MSVC (èíèöèàëèçàöèÿ security cookie, êîíòðîëü ïðåîáðàçîâàíèÿ ñòåê, Control Flow Guard), à íå ê ñïåöèàëüíî íàïèñàííûì àâòîðàìè ïðîãðàììû ïðîâåðêàì. Ïðèíàäëåæíîñòü

êàæäîãî êîíêðåòíîãî àäðåñà ïîëüçîâàòåëüñêîìó êîäó èëè êîäó CRT óòî÷íÿåòñÿ ïî ãðàôó âûçîâîâ â IDA Pro.



Рис. 2. IDA Pro: обращение к PEB через fs:[0x30] (Proximity view / дизассемблер)



Рис. 3. IDA Pro: инструкция int 0x2D и соседний код

4.3. Фрагмент кода проверки и вывода результата

```
401c13  push   0x18f
401c18  push   0x0
401c1a  push   0x92e7
401c1f  push   0x30
401c21  push   0x401570
401c26  call   0x401710          ; декодирование строки сообщения
401c2b  add    esp,0x14
401c2e  lea    ecx,[ebp-0x130]
401c34  push   ecx
401c35  call   0x401570          ; проверка пароля
401c3a  add    esp,0x4
401c3d  test   eax,eax
401c3f  je     0x401c7a          ; переход на ветку "Access
denied!"
...
401c59  push   0xa
401c5b  lea    ecx,[ebp-0x130]
401c61  push   ecx
401c62  call   0x4017a0          ; вычисление "секретного кода"
401c67  add    esp,0x4
401c6a  push   eax
401c6b  push   0x41ef0c          ; формат-строка вывода
401c70  call   0x401050          ; вывод в консоль
401c75  add    esp,0xc
```

Функция sub_4017A0 суммирует коды всех символов введённой строки и возвращает сумму в eax — это и есть «секретный код», выводимый программой при успешной проверке.

```
4017ad  mov    eax,[ebp+0x8]        ; указатель на строку
4017b3..4017cc                      ; вычисление длины строки
(strlen)
4017dd  mov    [ebp-0xc],0x0        ; i = 0
4017f7  mov    edx,[ebp+0x8]
4017fa  add    edx,[ebp-0xc]
4017fd  movsx  eax,byte [edx]       ; очередной символ
401800  add    eax,[ebp-0x10]       ; сумма += символ
401803  mov    [ebp-0x10],eax
401808  mov    eax,[ebp-0x10]       ; return сумма
```

Для полной введённой строки `SuperSebretBackdnnrPassvnrd0102446789k` (пароль, найденный символьным выполнением, см. далее, плюс завершающий символ `k`, который требуется функцией `main`) сумма кодов символов равна **3420** (`0xD5C`). Это предварительное, статически вычисленное значение «секретного кода» — оно требует подтверждения фактическим запуском программы, так как содержимое форматной строки `0x41EF0C` не было декодировано статически.



Рис. 4. Отладчик/IDA: точка останова на 0x401c62 и значение eax после возврата из sub_4017a0 (секретный код)



Рис. 5. Консоль программы: вывод Success! и секретного кода после ввода найденного пароля

5. Скрипт IDAPython для поиска механизмов самозащиты

Для автоматизации поиска перечисленных выше признаков был написан скрипт `ida_script.py` (полный текст — Приложение А). Скрипт обходит все небиблиотечные функции базы и для каждой инструкции проверяет принадлежность к одной из категорий:

- `find_xor_crypto` — ищет внутри циклов инструкции `xor reg, reg2` (не являющиеся обнулением регистра), то есть потенциальные побайтовые операции шифрования/декодирования;
- `find_byte_strings` — ищет цепочки инструкций `mov [mem], imm8`, записывающих печатные ASCII-байты подряд — характерный признак посимвольно «зашитой» в код строки;
- `find_data_blobs` — находит блоки данных без распознанного типа (`unk_*`), на которые ссылается код, с фильтрацией мусора (заливка, импорт, уже распознанные строки);
- `find_peb_accesses` — по регулярному выражению ищет в дизассемблированной строке инструкции обращения `fs:`/`gs:` и сопоставляет смещение с известным расположением PEB;
- `find_time_checks` — ищет инструкции `rdtsc`/`rdtscp`, а также вызовы Time-API (`GetTickCount`, `QueryPerformanceCounter` и т. п.);
- `find_rare_instructions` — ищет редкие/привилегированные инструкции (`int`, `cpuid`, `sidt`, `rdmsr` и др.) и обращения к сегментным регистрам;
- `find_protection_indicators` — ищет вызовы типичных anti-debug/anti-VM API (`IsDebuggerPresent`, `VirtualProtect`, `CheckRemoteDebuggerPresent` и т. д.) и строки-маркеры виртуальных машин.

Скрипт запускается в контексте уже открытой в IDA Pro базы (`File → Script file...`) и выводит результаты построчно в Output window, попутно разбивая их по функциям.



Рис. 6. Output window IDA Pro: результат работы ida_script.py на crackme32.exe (полный лог)

6. Символьное выполнение основного файла (angr)

6.1. Идея решения

Ручной анализ цикла сравнения по адресу `0x4016C0` показал, что в наивной постановке символьное выполнение этого цикла породило бы до 2^{37} путей (на каждой итерации сравнения происходит условный переход и, соответственно, разделение состояния). Чтобы избежать комбинаторного взрыва, вместо прямого прохождения цикла в код установлен хук (`angr.Project.hook`) на адрес начала цикла сравнения: вместо эмуляции самого цикла в решатель сразу добавляются 37 ограничений вида `var_2C[i] == var_54[i]`, после чего все переходы `cmp`/`jz` внутри цикла становятся тривиально истинными и символьное выполнение идёт по единственной ветке до `ret` с `eax == 1`.

Скрипт `angr_solver.py` (Приложение Б):

1. загружает `crackme32.exe` как `angr.Project`;
2. устанавливает хук-замену цикла сравнения на адресе `0x4016C0`;
3. создаёт символьный буфер пароля (40 печатных ASCII-символов), ограниченный множеством `[0-9A-Za-z]`;
4. запускает `call_state` с точки входа `sub_401570` и ищет состояние, достигающее фиктивного адреса возврата `0x7FFFFFFF` (это гарантирует, что исследуется ровно один вызов функции без дальнейшего выполнения остального кода программы);
5. среди найденных состояний проверяет выполнимость условия `eax == 1` и извлекает конкретные байты пароля из решателя.

6.2. Результат

Скрипт был запущен на реальном файле `crackme32.exe`. Фактический вывод:

```
[*] Загружаю бинарник: ./crackme32.exe
[*] Хук на 0x4016c0 -- заменяем цикл на constraint
[*] Символьное исполнение sub_401570 @ 0x401570...
[*] Хук сработал раз: 37
[+] Состояние на ret достигнуто, eax == 1
```

```
[+] Пароль найден:
    hex  :
537570657253656272657442616 36b646e6e7250617373766e726430313032343436373839
    ascii: SuperSebretBackdnnrPassvnrd0102446789

    Полная строка для ввода (main требует хвостовой 'k'):
        SuperSebretBackdnnrPassvnrd0102446789k
```

Таким образом, пароль, принимаемый функцией `sub_401570`, — `SuperSebretBackdnnrPassvnrd0102446789` (37 символов); для прохождения проверки на уровне `main` ко вводу требуется добавить замыкающий символ `k`.



Рис. 7. Консоль: запуск angr_solver.py и найденный пароль

7. Обход защитных механизмов путём модификации программы

Обход реализован статическим патчем исполняемого файла (изменением байт непосредственно в файле на диске, без перехвата функций сторонних библиотек вроде mhook/minhook и без запуска под отладчиком). Патч был выполнен в hex-редакторе/IDA Pro (Edit → Patch program) и подтверждён побайтовым сравнением оригинального и модифицированного файлов.

В `crackme32.exe` изменены два места — оба относятся не к самой проверке пароля, а к функции анти-отладочного замера времени (двойной `rdtsc` и аварийное деление на ноль `div ch` при слишком большой задержке между замерами, см. Таблица 4):

Адрес (VA)	**Было**	**Стало**	**Инструкция была → стала**	**Эффект**
`0x4011C0`	`55 8B`	`C3 75`	`push ebp; mov ebp,esp` → `ret; ...`	Функция замера времени завершается сразу при входе, не доходя до измерений `rdtsc`
`0x401244`	`76`	`EB`	`jbe 0x401252` → `jmp 0x401252`	Безусловный пропуск аварийного `div ch` независимо от измеренной задержки

Таблица 4. Байтовые патчи в `crackme32.exe`

Первый патч превращает функцию проверки времени в no-op прямо на входе (`push ebp` заменён на `ret`), второй — дополнительно нейтрализует условный переход перед аварийным делением на ноль внутри той же функции на `jmp`. Два патча одновременно потребовались, по-видимому, потому что функция имеет вторую точку входа (переход `0x40121E → 0x401237` и блок `0x401226`,

характерные для компилятор-генерируемого SEH-фрейма `__try`/`__except`), с которой выполнение может попадать в код замера времени в обход патченного пролога.

В `crackme32_1.exe` (файл из `bin2`) изменены два байта внутри функции `sub_401000`, отвечающей за срабатывание access violation, перехватываемого затем SEH-обработчиком (см. раздел «Анализ файла из архива bin2»):

Адрес (VA)	**Было**	**Стало**	**Инструкция была → стала**
`0x401026`	`C7 45 FC 00 00 00 00`	`90 45 FC 00 00 00 00`	`mov dword ptr [ebp-4], 0` → `nop` + «мусорные» байты (`inc ebp`, `cld`, `add [eax],al`×2)
`0x40102D`	`C7 05 60 41 41 00 FF FF FF FF`	`90 05 60 41 41 00 FF FF FF FF`	`mov dword ptr ds:0x414160, 0xFFFFFFFF` → `nop` + «мусорные» байты (`add eax,0x414160`, далее некорректные опкоды)

Таблица 5. Байтовые патчи в `crackme32_1.exe`

В обоих случаях патч сводится к замене только первого байта опкода инструкции `mov dword ptr ..., imm32` (`C7`) на `90` (`nop`). Самое важное изменение — патч по адресу `0x40102D`: именно эта инструкция в оригинале записывает значение по адресу `0x414160`, лежащему в секции `.rdata` (только для чтения), и тем самым намеренно вызывает исключение доступа. После патча запись не происходит, исключение не возникает, и SEH-обработчик, ранее перехватывавший эту ошибку и перенаправлявший поток управления на `0x40122A`, больше не задействуется — код просто линейно продолжает выполнение. Оставшиеся байты патченных инструкций (не относящиеся к самому опкоду) не были изменены и после патча декодируются процессором как самостоятельные «мусорные» инструкции (например, `inc ebp`), но, судя по результату тестирования, на корректность работы программы это не повлияло.



Рис. 8. Патч 55 8B → C3 75 по адресу 0x4011c0 в hex-редакторе/IDA (Patch program)



Рис. 9. Патч 76 → EB по адресу 0x401244 в hex-редакторе/IDA (Patch program)



Рис. 10. Запуск модифицированного crackme32.exe с произвольным (не настоящим) паролем — защита по времени не срабатывает



Рис. 11. Патчи 0x401026 и 0x40102d в hex-редакторе/IDA для crackme32_1.exe



Рис. 12. Запуск модифицированного crackme32_1.exe — SEH-ловушка не срабатывает

8. Анализ файла из архива `bin2` (`crackme32_1.exe`)

8.1. Алгоритм и защитные механизмы

Файл `crackme32_1.exe` построен иначе: ключевая логика скрыта за структурным обработчиком исключений (SEH) — техника, иногда называемая exception-oriented obfuscation. По адресу `0x40120B` выполняется запись `mov dword ptr ds:0x414160, 1` — при этом адрес `0x414160` лежит в начале секции `.rdata`, доступной только на чтение, поэтому эта инструкция гарантированно вызывает исключение доступа (`STATUS_ACCESS_VIOLATION`). Исключение перехватывается заранее установленным SEH-обработчиком, который переводит выполнение не на следующую по порядку инструкцию, а на адрес `0x40122A`. Аналогичный приём применён ещё в двух местах (`0x401292 → 0x4012FA`, `0x4012A5 → 0x4012C1`).

Эффект двойной: во-первых, линейный дизассемблер без эмуляции SEH-цепочки покажет неверный (мёртвый) код после каждой такой инструкции; во-вторых, отладчик при пошаговом выполнении остановится на самом исключении, что также служит индикатором отладки (штатное исключение в защите или признак постороннего вмешательства). Строка `"Oh no! A password exception has occurred!"`, найденная в файле, подтверждает, что генерация исключения — осознанный элемент логики проверки, а не ошибка.

Проверка пароля выполняется функцией `sub_401000`, вызываемой из `sub_401190`; функция сравнивает побайтово две половины введённой строки (первые и вторые 7 из 14 значащих символов) с двумя эталонными 7-байтовыми массивами (`aJU129`, `a0iDswq`), расположенными в данных программы.

Место для рисунка

Рис. 13. IDA Pro: граф функции sub_401190 с переходами через SEH-обработчик (красные/разорванные рёбра)

8.2. Символьное выполнение и решение

Из-за SEH-ловушек прямое символьное выполнение `sub_401190` через angr не работает штатно — framework не подхватывает перенаправление потока управления исключением так, как это делает ОС. Решение, реализованное в `angr_bin2.py` (Приложение В):

1. на каждый из трёх адресов, вызывающих исключение, установлен хук, который просто присваивает `state.regs.eip` адрес соответствующего SEH-обработчика — тем самым имитируется эффект обработки исключения без его реального возбуждения;
2. сама функция сравнения `sub_401000` целиком заменена на `SimProcedure` на Python, который напрямую добавляет в решатель требуемые равенства между байтами введённой строки и байтами двух эталонных массивов, минуя посимвольное исполнение;
3. дальше, как и в первом скрипте, выполнение запускается с `call_state` на `sub_401190` и ищется состояние, достигающее фиктивного адреса возврата.

Фактический вывод скрипта на реальном файле:

```
[*] SEH-хук: 0x40120b → 0x40122a
[*] SEH-хук: 0x401292 → 0x4012fa
[*] SEH-хук: 0x4012a5 → 0x4012c1
[*] sub_401000 → SimProcedure
[*] Ищу путь до ret...
  [SEH] 0x40120b → 0x40122a
  [SEH] 0x401292 → 0x4012fa
[+] eax = 1
[+] Пароль (hex): 6a2a75403132396b622366776a31
[+] Пароль (ascii): j*u@129kb#fwj1
```

Таким образом, пароль для `crackme32_1.exe` — `j*u@129kb#fwj1`.

Место для рисунка

Рис. 14. Консоль: запуск angr_bin2.py и найденный пароль для файла из bin2

Место для рисунка

Рис. 15. Запуск crackme32_1.exe с найденным паролем — успешная проверка

9. Выводы

В ходе лабораторной работы изучены типовые методы самозащиты исполняемых файлов (проверка присутствия отладчика, обращение к PEB, замер времени выполнения, использование редких инструкций и структурных исключений как способа антидизассемблирования) и способы их выявления и обхода. Разработан скрипт IDAPython, автоматически находящий перечисленные признаки защиты в дизассемблированном коде. С использованием фреймворка символьного выполнения angr получены пароли для основного исследуемого файла (`SuperSebretBackdnnrPassvnrd0102446789k`) и для файла из архива `bin2` (`j*u@129kb#fwj1`), а также статически определено место и способ вычисления «секретного кода», выводимого программой при успешной проверке. Предложен способ обхода защиты как путём статического патча условного перехода, так и путём динамического перехвата защитных API.

Приложения

Приложение А

ida_script.py

```
import idautils
import idc
import ida_funcs
import ida_gdl
import ida_ua
import ida_bytes
import ida_segment
import re

MIN_BLOB_LEN = 8        # минимальная длина блока для вывода
MAX_BLOB_LEN = 0x1000   # ограничение на длину блока
SKIP_STRLIT = False      # True - пропускать то, что IDA уже
определила как строку
NAME_PREFIXES = ("unk_", "byte_")   # брать только блоки с такими
авто-именами
                                    # (None - не фильтровать по
имени)
ONLY_READONLY = False    # только неизменяемые секции (.rdata),
без .data/.bss
MIN_UNIQUE_BYTES = 6    # минимум разных байт (отсекает 00 00.. /
FF FF.. и т.п.)

PEB_OFFSETS = {
    ("fs", 0x30): "PEB (Windows x86)",
    ("gs", 0x60): "PEB (Windows x64)",
}

TIME_API_NAMES = {
    "gettickcount": "GetTickCount",
    "gettickcount64": "GetTickCount64",
    "queryperformancecounter": "QueryPerformanceCounter",
    "queryperformancefrequency": "QueryPerformanceFrequency",
    "getsystemtime": "GetSystemTime",
    "getsystemtimeasfiletime": "GetSystemTimeAsFileTime",
```

```
    "getsystemtimepreciseasfiletime":
"GetSystemTimePreciseAsFileTime",
    "timegettime": "timeGetTime",
    "ntquerysystemtime": "NtQuerySystemTime",
    "ntqueryperformancecounter": "NtQueryPerformanceCounter",
    "zwquerysystemtime": "ZwQuerySystemTime",
    "clock_gettime": "clock_gettime",
    "time": "time",
    "clock": "clock",
}

TIME_INSTRUCTIONS = {
    "rdtsc": "RDTSC",
    "rdtscp": "RDTSCP",
}


RARE_INSTRUCTIONS = {
    "int": "Software interrupt",
    "int1": "ICEBP / INT1",
    "int3": "Breakpoint interrupt",
    "into": "Overflow interrupt (x86)",
    "icebp": "ICEBP",
    "ud0": "Undefined instruction",
    "ud1": "Undefined instruction",
    "ud2": "Undefined instruction",
    "hlt": "Halt",
    "cli": "Disable interrupts",
    "sti": "Enable interrupts",
    "in": "Port input",
    "out": "Port output",
    "ins": "String port input",
    "outs": "String port output",
    "sidt": "Store IDT",
    "sgdt": "Store GDT",
    "sldt": "Store LDT",
    "str": "Store task register",
    "smsw": "Store machine status word",
    "lldt": "Load LDT",
    "ltr": "Load task register",
```

```
    "lgdt": "Load GDT",
    "lidt": "Load IDT",
    "invlpg": "Invalidate TLB page",
    "invd": "Invalidate cache",
    "wbinvd": "Write back and invalidate cache",
    "syscall": "System call",
    "sysret": "System return",
    "sysenter": "Fast system call",
    "sysexit": "Fast system return",
    "vmcall": "Virtual machine call",
    "vmmcall": "Virtual machine call",
    "vmrun": "Enter VMX guest",
    "vmlaunch": "Launch VMX guest",
    "vmresume": "Resume VMX guest",
    "vmxoff": "Leave VMX operation",
    "xbegin": "Transactional execution",
    "xend": "Transactional execution end",
    "xtest": "Transactional execution test",
    "lar": "Load access rights",
    "lsl": "Load segment limit",
    "verr": "Verify segment for reading",
    "verw": "Verify segment for writing",
    "arpl": "Adjust RPL (x86)",
    "bound": "Bounds check (x86)",
    "aam": "ASCII adjust after multiply",
    "aad": "ASCII adjust before divide",
    "daa": "Decimal adjust after addition",
    "das": "Decimal adjust after subtraction",
    "aaa": "ASCII adjust after addition",
    "aas": "ASCII adjust after subtraction",
    "rsm": "Resume from SMM",
    "rdmsr": "Read model-specific register",
    "wrmsr": "Write model-specific register",
    "rdpmc": "Read performance counter",
    "xgetbv": "Read extended control register",
    "xsetbv": "Write extended control register",
    "cpuid": "CPU identification",
}

SEGMENT_REGISTERS = {
```

```
    "cs", "ds", "es", "ss"
}


PROTECTION_APIS = {
    "IsDebuggerPresent":
        "Проверка наличия отладчика",

    "CheckRemoteDebuggerPresent":
        "Проверка удалённого отладчика",

    "NtQueryInformationProcess":
        "Получение информации о процессе",

    "OutputDebugStringA":
        "Взаимодействие с отладчиком",

    "OutputDebugStringW":
        "Взаимодействие с отладчиком",

    "SetUnhandledExceptionFilter":
        "Установка обработчика исключений",

    "AddVectoredExceptionHandler":
        "Регистрация обработчика исключений",

    "RemoveVectoredExceptionHandler":
        "Удаление обработчика исключений",

    "RaiseException":
        "Генерация исключения",

    "VirtualProtect":
        "Изменение защиты памяти",

    "VirtualProtectEx":
        "Изменение защиты памяти другого процесса",

    "VirtualAlloc":
        "Выделение памяти",
```

```
    "VirtualAllocEx":
        "Выделение памяти другого процесса",

    "NtProtectVirtualMemory":
        "Изменение защиты памяти через NT API",

    "NtAllocateVirtualMemory":
        "Выделение памяти через NT API",

    "FlushInstructionCache":
        "Сброс кэша инструкций",

    "GetModuleHandleA":
        "Получение дескриптора модуля",

    "GetModuleHandleW":
        "Получение дескриптора модуля",

    "GetProcAddress":
        "Динамическое разрешение API",

    "LoadLibraryA":
        "Динамическая загрузка библиотеки",

    "LoadLibraryW":
        "Динамическая загрузка библиотеки",
}


VM_MARKERS = (
    "vmware",
    "virtualbox",
    "vbox",
    "qemu",
    "xen",
    "hyper-v",
    "hypervisor",
)
```

```
def is_library(f):
    return bool(f.flags & (ida_funcs.FUNC_LIB |
ida_funcs.FUNC_THUNK))


def get_loop_blocks(func):
    """Возвращает список циклов (каждый цикл - множество
блоков)."""
    fc = ida_gdl.FlowChart(func, flags=ida_gdl.FC_PREDS)
    loops = []
    for tail in fc:
        for head in tail.succs():
            if head.start_ea <= tail.start_ea:  # обратное ребро
                body = {head.id: head, tail.id: tail}
                stack = [tail]

                while stack:
                    b = stack.pop()

                    if b.id == head.id:
                        continue

                    for p in b.preds():
                        if p.id not in body and p.start_ea >=
head.start_ea:
                            body[p.id] = p
                            stack.append(p)

                loops.append(list(body.values()))

    return loops


def is_useful_xor(ea):
    """xor, но не обнуление регистра (xor eax, eax)."""
    if idc.print_insn_mnem(ea) != "xor":
        return False

    insn = ida_ua.insn_t()
```

```
    if ida_ua.decode_insn(insn, ea) == 0:
        return False

    a, b = insn.ops[0], insn.ops[1]
    if a.type == ida_ua.o_reg and b.type == ida_ua.o_reg and a.reg
== b.reg:
        return False

    return True


def find_xor_crypto():
    funcs_xor = {}

    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)
        if not f or is_library(f):
            continue

        for loop in get_loop_blocks(f):
            for blk in loop:
                for ea in idautils.Heads(blk.start_ea,
blk.end_ea):
                    if is_useful_xor(ea):
                        funcs_xor.setdefault(fea, set()).add(ea)

    return {fea: sorted(eas) for fea, eas in funcs_xor.items()}

def find_byte_strings():
    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)
        if not f or is_library(f):
            continue

        chain = []

        for ea in idautils.Heads(f.start_ea, f.end_ea):
            b = _get_printable_byte(ea)
            if b is not None:
                chain.append((ea, b))
```

```
                continue

            if len(chain) >= 4:
                _print_chain(f, chain)
            chain = []

        if len(chain) >= 4:
            _print_chain(f, chain)


def _get_printable_byte(ea):
    if idc.print_insn_mnem(ea) != "mov":
        return None

    insn = ida_ua.insn_t()
    if ida_ua.decode_insn(insn, ea) == 0:
        return None
    if len(insn.ops) < 2:
        return None

    dst, src = insn.ops[0], insn.ops[1]

    if dst.type != ida_ua.o_displ:
        return None
    if src.type != ida_ua.o_imm:
        return None

    v = src.value & 0xFF
    if not (32 <= v < 127):
        return None

    return v


def _print_chain(f, chain):
    text = "".join(chr(b) for _, b in chain)
    print("[byte-str] func 0x%X  len=%d  %r"
          % (f.start_ea, len(chain), text))
```

```
def _is_data_target(ea):
    """Адрес лежит в неисполняемом сегменте и не размечен как
код."""
    seg = ida_segment.getseg(ea)
    if not seg or (seg.perm & ida_segment.SEGPERM_EXEC):
        return False
    return not ida_bytes.is_code(ida_bytes.get_flags(ea))


def read_blob(start):
    """Читает байты от метки start до следующей метки/инструкции/
конца сегмента."""
    seg = ida_segment.getseg(start)
    if not seg:
        return b""

    data = bytearray()
    cur = start

    while cur < seg.end_ea and len(data) < MAX_BLOB_LEN:
        flags = ida_bytes.get_flags(cur)

        if ida_bytes.is_code(flags):
            break

        # следующая метка или объект с xref - граница блока
        if cur != start and (ida_bytes.has_any_name(flags) or
ida_bytes.has_xref(flags)):
            break

        size = ida_bytes.get_item_size(cur) or 1
        chunk = ida_bytes.get_bytes(cur, size)
        if chunk is None:
            break

        data += chunk
        cur += size

    return bytes(data)
```

```
def is_interesting_blob(ea, data):
    """Отсев мусора: импорты, заливка, указатели, структуры,
готовые строки."""
    flags = ida_bytes.get_flags(ea)

    if NAME_PREFIXES:
        if not idc.get_name(ea).startswith(NAME_PREFIXES):
            return False

    if ONLY_READONLY:
        seg = ida_segment.getseg(ea)
        if seg and (seg.perm & ida_segment.SEGPERM_WRITE):
            return False

    if SKIP_STRLIT and ida_bytes.is_strlit(flags):
        return False
    if ida_bytes.is_off0(flags) or ida_bytes.is_struct(flags):
        return False

    if len(set(data)) < MIN_UNIQUE_BYTES:
        return False

    return True


def find_data_blobs():
    """Ищет блоки данных (unk_XXXX и т.п.), на которые ссылаются
небиблиотечные функции."""
    blobs = {}

    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)
        if not f or is_library(f):
            continue

        for head in idautils.Heads(f.start_ea, f.end_ea):
            if not ida_bytes.is_code(ida_bytes.get_flags(head)):
                continue
```

```
            for xr in idautils.XrefsFrom(head, 0):
                if xr.iscode:
                    continue
                if not _is_data_target(xr.to):
                    continue
                blobs.setdefault(xr.to, []).append((fea, head))

    for ea in sorted(blobs):
        data = read_blob(ea)
        if len(data) < MIN_BLOB_LEN:
            continue
        if not is_interesting_blob(ea, data):
            continue

        ascii_repr = "".join(chr(b) if 32 <= b < 127 else "." for
b in data)
        print("[blob] 0x%X  %s  len=%d" % (ea, idc.get_name(ea),
len(data)))
        print("       hex  : %s" % " ".join("%02X" % b for b in
data))
        print("       ascii: %s" % ascii_repr)

        for fea, xea in blobs[ea]:
            print("       xref : %s (0x%X) <- 0x%X"
                  % (idc.get_func_name(fea), fea, xea))

def find_peb_accesses():
    results = {}

    pattern = re.compile(
        r"\b(fs|gs)\s*:\s*"
        r"(?:\[\s*)?"
        r"(0x[0-9a-f]+|[0-9a-f]+h?)\b",
        re.IGNORECASE
    )

    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)

        if not f or is_library(f):
```

```
            continue

        for ea in idautils.Heads(f.start_ea, f.end_ea):
            if not ida_bytes.is_code(ida_bytes.get_flags(ea)):
                continue

            disasm = idc.generate_disasm_line(ea, 0)
            if not disasm:
                continue

            matches = list(pattern.finditer(disasm))
            if not matches:
                continue

            descriptions = []

            for match in matches:
                segment = match.group(1).lower()
                raw_offset = match.group(2).lower()

                if raw_offset.startswith("0x"):
                    offset = int(raw_offset, 16)
                elif raw_offset.endswith("h"):
                    offset = int(raw_offset[:-1], 16)
                else:
                    offset = int(raw_offset, 16)

                desc = PEB_OFFSETS.get((segment, offset))

                if desc:
                    descriptions.append(desc)
                else:
                    descriptions.append(
                        "FS/GS access: %s:[0x%X]"
                        % (segment.upper(), offset)
                    )

            results.setdefault(fea, []).append(
                (ea, "; ".join(descriptions), disasm)
            )
```

```
    return results


def print_peb_accesses():
    results = find_peb_accesses()

    total = 0

    for fea in sorted(results):
        accesses = results[fea]

        print(
            "\n[+] %s | 0x%X"
            % (idc.get_func_name(fea), fea)
        )

        for ea, description, disasm in accesses:
            print(
                "    0x%X  [%s]  %s"
                % (ea, description, disasm)
            )
            total += 1

    print("\nНайдено обращений: %d" % total)

    return results

def find_time_checks():
    results = {}

    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)

        if not f or is_library(f):
            continue

        for ea in idautils.Heads(f.start_ea, f.end_ea):
            if not ida_bytes.is_code(ida_bytes.get_flags(ea)):
                continue
```

```
            mnem = idc.print_insn_mnem(ea).lower()
            disasm = idc.generate_disasm_line(ea, 0) or
idc.GetDisasm(ea)

            category = None
            description = None

            if mnem in TIME_INSTRUCTIONS:
                category = "CPU counter"
                description = TIME_INSTRUCTIONS[mnem]

            elif mnem in ("call", "callq", "bl", "blr"):
                target_name = idc.get_name(
                    idc.get_operand_value(ea, 0)
                ).lower()

                normalized = target_name.split(".")[-1]
                normalized = normalized.lstrip("_")
                normalized = normalized.split("@")[0]

                if normalized in TIME_API_NAMES:
                    category = "Time API"
                    description = TIME_API_NAMES[normalized]

                else:
                    text = disasm.lower()

                    for api_name, canonical_name in
TIME_API_NAMES.items():
                        if re.search(
                            r"(?<![a-z0-9_])"
                            + re.escape(api_name)
                            + r"(?![a-z0-9_])",
                            text
                        ):
                            category = "Time API"
                            description = canonical_name
                            break
```

```
            if category:
                results.setdefault(fea, []).append(
                    (ea, category, description, disasm)
                )

    return results


def print_time_checks():
    results = find_time_checks()
    total = 0

    for fea in sorted(results):
        print(
            "\n[+] %s | 0x%X"
            % (idc.get_func_name(fea), fea)
        )

        for ea, category, description, disasm in results[fea]:
            print(
                "    0x%X  [%s] %s | %s"
                % (ea, category, description, disasm)
            )
            total += 1

    print("\nНайдено мест: %d" % total)

    return results

def find_rare_instructions():
    results = {}

    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)

        if not f or is_library(f):
            continue

        for ea in idautils.Heads(f.start_ea, f.end_ea):
            if not ida_bytes.is_code(ida_bytes.get_flags(ea)):
```

```
                continue

            mnem = idc.print_insn_mnem(ea).lower()
            disasm = idc.generate_disasm_line(ea, 0) or
idc.GetDisasm(ea)

            category = None
            description = None

            if mnem in RARE_INSTRUCTIONS:
                category = "Rare instruction"
                description = RARE_INSTRUCTIONS[mnem]

                if mnem == "int":
                    insn = ida_ua.insn_t()

                    if ida_ua.decode_insn(insn, ea):
                        op = insn.ops[0]

                        if op.type == ida_ua.o_imm:
                            vector = op.value

                            if vector == 0x2D:
                                description = "INT 2D (anti-debug
candidate)"
                            elif vector == 0x03:
                                description = "INT 3 (breakpoint)"
                            elif vector == 0x01:
                                description = "INT 1 (debug
exception)"
                            else:
                                description = "INT 0x%X" % vector

            segment_hits = []

            for reg in SEGMENT_REGISTERS:
                if re.search(
                    r"(?<![a-z0-9_])" + reg + r"(?![a-z0-9_])",
                    disasm.lower()
                ):
```

```
                    segment_hits.append(reg.upper())

            if segment_hits:
                seg_desc = "Segment register: " + ", ".join(
                    sorted(set(segment_hits))
                )

                if category:
                    description += "; " + seg_desc
                else:
                    category = "Segment register"
                    description = seg_desc

            if category:
                results.setdefault(fea, []).append(
                    (ea, category, description, disasm)
                )

    return results


def print_rare_instructions():
    results = find_rare_instructions()
    total = 0

    for fea in sorted(results):
        print(
            "\n[+] %s | 0x%X"
            % (idc.get_func_name(fea), fea)
        )

        for ea, category, description, disasm in results[fea]:
            print(
                "    0x%X  [%s] %s | %s"
                % (ea, category, description, disasm)
            )
            total += 1

    print("\nНайдено инструкций и обращений: %d" % total)
```

```
    return results

def find_protection_indicators():
    results = {}

    for fea in idautils.Functions():
        f = ida_funcs.get_func(fea)

        if not f or is_library(f):
            continue

        for ea in idautils.Heads(f.start_ea, f.end_ea):
            if not ida_bytes.is_code(ida_bytes.get_flags(ea)):
                continue

            mnem = idc.print_insn_mnem(ea).lower()
            disasm = (
                idc.generate_disasm_line(ea, 0)
                or idc.GetDisasm(ea)
            )

            text = disasm.lower()
            found = []

            if mnem in ("call", "callq", "jmp"):
                for api, description in PROTECTION_APIS.items():
                    if re.search(
                        r"(?<![a-z0-9_])"
                        + re.escape(api.lower())
                        + r"(?![a-z0-9_])",
                        text
                    ):
                        found.append((
                            "Protection API",
                            api + ": " + description
                        ))

            if mnem == "cpuid":
                found.append((
                    "Environment check",
```

```
                    "CPUID: проверка характеристик CPU"
                ))

            for marker in VM_MARKERS:
                if marker in text:
                    found.append((
                        "VM marker",
                        "Маркер виртуальной среды: " + marker
                    ))

            if found:
                entries = results.setdefault(fea, [])

                for category, description in found:
                    item = (ea, category, description, disasm)

                    if item not in entries:
                        entries.append(item)

    return results


def print_protection_indicators():
    results = find_protection_indicators()
    total = 0

    for fea in sorted(results):
        print(
            "\n[+] %s | 0x%X"
            % (idc.get_func_name(fea), fea)
        )

        for ea, category, description, disasm in results[fea]:
            print(
                "    0x%X  [%s] %s | %s"
                % (ea, category, description, disasm)
            )
            total += 1

    print("\nНайдено совпадений: %d" % total)
```

```
    print("\nПоиск строк виртуальных сред")

    string_count = 0

    for s in idautils.Strings():
        value = str(s).lower()

        for marker in VM_MARKERS:
            if marker in value:
                print(
                    "    0x%X  [VM string] %s"
                    % (s.ea, str(s))
                )
                string_count += 1
                break

    print("Найдено строк: %d" % string_count)

    return results

print("Поиск XOR")

funcs_xor = find_xor_crypto()
for fea in funcs_xor.keys():
    print("[+] %s | 0x%X" % (idc.get_func_name(fea), fea))

    for ea in funcs_xor[fea]:
        print("      0x%X  %s" % (ea, idc.GetDisasm(ea)))

print("\nПоиск XOR-строк")

find_byte_strings()

print("\nПоиск блоков данных (unk_*)")

find_data_blobs()

print("\nПоиск обращений к PEB")
print_peb_accesses()
```

```
print("\nПоиск мест замера времени")
print_time_checks()

print("\nПоиск редких инструкций")
print_rare_instructions()

print("\nПоиск дополнительных признаков защиты")
print_protection_indicators()

print("Поиск окончен")
```

Приложение Б

angr_solver.py

```
import sys
import angr
from angr import claripy

#
--------------------------------------------------------------------------------
# НАСТРОЙКИ
#
--------------------------------------------------------------------------------
BINARY_PATH   = "./crackme32.exe"
FUNC_ADDR     = 0x00401570
BUF_LEN       = 40
PASSWORD_LEN  = 37
FAKE_RET_ADDR = 0x7FFFFFFF
BUF_ADDR      = 0x70000000   # любой свободный адрес

# Адреса из IDA
COMPARE_LOOP_ADDR = 0x004016C0   # начало цикла сравнения
(loc_4016C0)


def main():
    print(f"[*] Загружаю бинарник: {BINARY_PATH}")
    proj = angr.Project(BINARY_PATH, auto_load_libs=False)

    #
-------------------------------------------------------------------
    # Хук на начало цикла сравнения.
    #
    # Оригинал делает 37 раз:
    #     movzx ecx, [ebp+eax+var_2C]   ; symbol
    #     movzx eax, [ebp+edx+var_54]   ; concrete
    #     cmp   ecx, eax
    #     jz    ...
    # и на каждом шаге форкает состояние -> до 2**37 путей.
    #
```

```
    # Мы заменяем это одной constraint: var_2C[i] == var_54[i] для
всех i.
    # После этого все cmp в цикле становятся тривиально истинными,
    # angr идёт по одной ветке до "mov eax, 1".
    #
----------------------------------------------------------------------
    hook_hits = [0]

    def hook_compare(state):
        hook_hits[0] += 1
        ebp_val = state.regs.ebp
        # var_2C: трансформированный ввод
        # var_54: эталон (скопирован из unk_41EE50)
        for i in range(PASSWORD_LEN):
            sym = state.memory.load(ebp_val - 0x2C + i, 1)
            tgt = state.memory.load(ebp_val - 0x54 + i, 1)
            state.solver.add(sym == tgt)

    proj.hook(COMPARE_LOOP_ADDR, hook_compare, length=0)
    print(f"[*] Хук на {hex(COMPARE_LOOP_ADDR)} -- заменяем цикл
на constraint")

    #
----------------------------------------------------------------------
    # Символьный буфер
    #
----------------------------------------------------------------------
    password_chars = [claripy.BVS(f"pw_{i}", 8) for i in
range(BUF_LEN)]

    state = proj.factory.call_state(
        FUNC_ADDR,
        BUF_ADDR,
        ret_addr=FAKE_RET_ADDR,
        prototype="int f(char*)",
        add_options={
            angr.options.ZERO_FILL_UNCONSTRAINED_MEMORY,
            angr.options.ZERO_FILL_UNCONSTRAINED_REGISTERS,
        },
    )
```

```
    # Записываем байты в память по одному -- правильный порядок
    for i in range(BUF_LEN):
        state.memory.store(BUF_ADDR + i, password_chars[i])

    # Ограничения: [0-9A-Za-z]
    for i in range(PASSWORD_LEN):
        c = password_chars[i]
        state.solver.add(
            claripy.Or(
                claripy.And(c >= ord('0'), c <= ord('9')),
                claripy.And(c >= ord('a'), c <= ord('z')),
                claripy.And(c >= ord('A'), c <= ord('Z')),
            )
        )
    state.solver.add(password_chars[PASSWORD_LEN] == 0)

    #
----------------------------------------------------------------------
    # Запуск
    #
----------------------------------------------------------------------
    print(f"[*] Символьное исполнение sub_401570 @
{hex(FUNC_ADDR)}...")
    simgr = proj.factory.simulation_manager(state)
    simgr.explore(find=FAKE_RET_ADDR, num_find=1)

    print(f"[*] Хук сработал раз: {hook_hits[0]}")

    if not simgr.found:
        print("[-] Не дошли до ret. Проверьте адрес хука / базу
загрузки.")
        sys.exit(1)

    solution = simgr.found[0]

    # Проверяем, что eax действительно может быть 1
    if not
solution.solver.satisfiable(extra_constraints=[solution.regs.eax
== 1]):
```

```
        print("[-] Состояние на ret имеет eax != 1.")
        print(f"    eax =
{solution.solver.eval(solution.regs.eax)}")
        sys.exit(1)

    solution.solver.add(solution.regs.eax == 1)
    print("[+] Состояние на ret достигнуто, eax == 1")

    #
----------------------------------------------------------------------
    # Извлекаем байты
    #
----------------------------------------------------------------------
    solved_bytes = b''.join(
        solution.solver.eval(solution.memory.load(BUF_ADDR + i,
1), cast_to=bytes)
        for i in range(BUF_LEN)
    )

    nul  = solved_bytes.find(b"\x00")
    core = solved_bytes[:nul] if nul != -1 else solved_bytes

    print()
    print("[+] Пароль найден:")
    print("    hex  :", core.hex())
    try:
        print("    ascii:", core.decode("ascii"))
    except UnicodeDecodeError:
        print("    (не ASCII)")

    try:
        full_input = core.decode("ascii") + "k"
        print()
        print("    Полная строка для ввода (main требует хвостовой
'k'):")
        print("       ", full_input)
    except UnicodeDecodeError:
        pass
```

```python
if __name__ == "__main__":
    main()
```

Приложение В

angr_bin2.py

```
import sys
import angr
from angr import claripy

# --- фикс bitstring 4.x ---
import bitstring
if not hasattr(bitstring, "ConstBitStream"):
    bitstring.ConstBitStream = bitstring.Bits
# --------------------------

BINARY_PATH = "./crackme32_1.exe"
SUB_401190  = 0x00401190
SUB_401000  = 0x00401000
BUF_ADDR    = 0x0041B020
PASSWORD_LEN = 14
FAKE_RET    = 0x7FFFFFFF

SEH_TRAMPOLINES = {
    0x0040120B: 0x0040122A,
    0x00401292: 0x004012FA,
    0x004012A5: 0x004012C1,
}


class Sub401000(angr.SimProcedure):
    """Полная замена sub_401000: добавляет constraint и возвращает
1."""
    def run(self, arg0, arg4):
        # arg0 = aJU129, arg4 = a0iDswq (оба — указатели)
        for i in range(7):
            b0 = self.state.memory.load(arg0 + i, 1)
            b4 = self.state.memory.load(arg4 + i, 1)
            inp_lo = self.state.memory.load(BUF_ADDR + i, 1)
            inp_hi = self.state.memory.load(BUF_ADDR + 7 + i, 1)
            self.state.solver.add(inp_lo == b0)
            self.state.solver.add(inp_hi == b4)
```

```
        return 1


def main():
    proj = angr.Project(BINARY_PATH, auto_load_libs=False)
    password_chars = [claripy.BVS(f"pw_{i}", 8) for i in
range(PASSWORD_LEN)]

    # --- SEH-хуки ---
    for addr, target in SEH_TRAMPOLINES.items():
        def make_hook(tgt):
            def hook(state):
                print(f"  [SEH]
{hex(state.solver.eval(state.regs.eip))} → {hex(tgt)}")
                state.regs.eip = tgt
            return hook
        proj.hook(addr, make_hook(target), length=10)
        print(f"[*] SEH-хук: {hex(addr)} → {hex(target)}")

    # --- Замена sub_401000 на SimProcedure (не length=0!) ---
    proj.hook(SUB_401000, Sub401000())
    print(f"[*] sub_401000 → SimProcedure")

    # --- Старт с sub_401190 ---
    state = proj.factory.call_state(
        SUB_401190,
        ret_addr=FAKE_RET,
        prototype="int f()",
        add_options={
            angr.options.ZERO_FILL_UNCONSTRAINED_MEMORY,
            angr.options.ZERO_FILL_UNCONSTRAINED_REGISTERS,
        },
    )
    for i in range(PASSWORD_LEN):
        state.memory.store(BUF_ADDR + i, password_chars[i])
    state.memory.store(BUF_ADDR + PASSWORD_LEN, claripy.BVV(0, 8))

    # Печатные ASCII
    for i in range(PASSWORD_LEN):
        c = password_chars[i]
```

```
            state.solver.add(claripy.And(c >= 0x20, c <= 0x7E))

    simgr = proj.factory.simulation_manager(state)
    print("[*] Ищу путь до ret...")
    simgr.explore(find=FAKE_RET, num_find=1)

    if not simgr.found:
        print("[-] Не найдено")
        print(f"    deadended: {len(simgr.deadended)}, errored:
{len(simgr.errored)}, active: {len(simgr.active)}")
        for s in simgr.errored[:5]:
            print("    error:", s.error)
        for s in simgr.deadended[:5]:
            print("    dead eip:", hex(s.solver.eval(s.regs.eip)),
"eax:", s.solver.eval(s.regs.eax))
        return

    sol = simgr.found[0]
    print("[+] eax =", sol.solver.eval(sol.regs.eax))
    pw = b"".join(sol.solver.eval(password_chars[i],
cast_to=bytes) for i in range(PASSWORD_LEN))
    print("[+] Пароль (hex):", pw.hex())
    print("[+] Пароль (ascii):", pw.decode("latin-1"))


if __name__ == "__main__":
    main()
```